

Suricata Dashboard

📅 Date	July 27, 2026
--------	---------------

▼ Setup

Project Genesis & Requirements

- **Context:** Shifted away from older frameworks (such as Flask/Vue setups) to build a clean, modern, real-time security operations dashboard using **Streamlit** on Ubuntu.
- **Environment Setup:**
 - Installed Python pip and virtual environment modules (`sudo apt install python3-pip python3-venv -y`).
 - Initialized an isolated workspace (`python3 -m venv venv` and activated via `source venv/bin/activate`).
 - Installed core dependencies: `streamlit` , `streamlit-autorefresh` , and `pandas` .
 - Configured Git for independent version tracking.

▼ Core Architecture & Data Pipeline

- **Data Source (`eve.json`):** Configured to tail Suricata's real-time structured log output located at `/var/log/suricata/eve.json` .
- **Incremental Parsing Logic:** Implemented file-pointer caching (`eve_offset.txt`) and `f.seek()` to ingest *only* new log lines as they are written, preventing full file re-scans and memory bloating.
- **Dual-Path Storage Shift:**
 - **Alerts (`sqlite3` backend):** Transitioned alert events from a volatile session list into a persistent local SQLite database (`alerts.db`). This ensures security alerts survive browser refreshes, restarts, and UI

reloads, complete with deduplication keys (`dedup_key`) to avoid duplicate logging.

- **Traffic (`deque` /Rolling Window):** Maintained a session-only rolling limit (`MAX_TRAFFIC_EVENTS = 3000`) for non-alert traffic logs (flows, stats, DNS) to keep memory footprint light.

▼ Dashboard Layout & Modular Codebase Structure

The project is organized into a clean module layout:

1. `app.py` (Main Entrypoint & UI Shell):

- Configures wide-layout styling, custom CSS security severity chips (`.sev-chip`), and timezone support (`Asia/Karachi`).
- Implements real-time UI updates using Streamlit's `@st.fragment(run_every=5)` decorator, keeping page updates localized without full-page reloads.
- Renders tab views for **Alerts** and **Traffic**, complete with CSV export buttons and detailed expander views.

2. `modules/alert_store.py` (Persistent Database Handler):

- Manages the SQLite database connection schema (`alerts` table).
- Handles safe extraction fields (`_safe_get`), insertion logic with duplicate prevention, review status changes (`new` , `reviewed` , `false_positive`), and data-frame conversions.

3. `modules/data_loader.py` (Parser & Rules Engine):

- Manages incremental file tailing, offset position saving, and row mapping.
- Parses system stats (`uptime` , kernel packet drops, decoder bytes).
- Loads the extended Suricata rule catalog (`suricata_et_rules.json`) via `@st.cache_data` .

4. `modules/filters.py` (Advanced Filtering System):

- Provides synchronized multiselect dropdowns for severities, protocols, categories, attack targets, interfaces, and directions.
- Integrates robust text-search inputs equipped with optional **Regular Expression (Regex)** evaluation and fallback safety error handling.
- Parses flexible numeric range selectors for ports and byte metrics (e.g., `1000-2000` or comma-separated lists).

5. `modules/charts.py` **(Visualizations):**

- Aggregates metrics to display a high-level **KPI row** (Total Alerts, High Severity counts, Unique Signatures, Traffic Events).
- Renders inline visual charts for **Severity Breakdown, Alerts Over Time** (resampled to min-intervals), and **Top Signatures**.